

Woxsen University

School of Technology



PROJECT REPORT

PROJECT 41

DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT SYSTEM FOR CAMPUS FACILITY MAINTENANCE REQUEST MANAGEMENT SYSTEM

Submitted by:

Pranav Arjun Nikam | 25WU0101085

Purva Derkar | 25WU0101102

Sharanya Mishra | 25WU0101126

Saathvik Mahalsa | 25WU0101115

Course: Database Management Systems

University: Woxsen University

Faculty: Dr. Kiran Mayee Adavala

Academic Year: 2026–2027

2. ABSTRACT

Campus facilities depend on timely maintenance of classrooms, laboratories, offices, and shared services. When issue reporting is scattered across verbal requests or informal messages, records can be missed, duplicated, or difficult to trace. This project presents a database-backed Campus Facility Maintenance Request Management System that organizes reports and their follow-up in one relational application. A user records the affected room, category, priority, title, and description. The application assigns a unique request number while MySQL retains the internal auto-increment request identifier. The distinction is important: in the inspected sample, request numbers REQ-2026-001 through REQ-2026-005 correspond to database request IDs 11 through 15, not IDs 1 through 5.

The database separates campus buildings and rooms, users, request categories, priorities, technicians, assignments, work logs, materials, material usage, status history, and feedback. These relations provide referential integrity through primary and foreign keys. Unique constraints prevent repeated request numbers, multiple technician assignments for the same request, and duplicate feedback for a request. Check constraints enforce domain rules for roles, technician availability, assignment status, work hours, material cost, usage quantity, priority response hours, and feedback rating. The web interface presents dashboard totals, searchable and filterable requests, request details, technician selection, status transitions, work and material entry, closure, feedback, and aggregate reports.

The prototype uses Python 3.12 and Flask for routing and server-side validation, Jinja2 for HTML rendering, HTML5 and CSS3 for the interface, a small JavaScript file for defaulting date inputs, and mysql-connector-python to communicate with MySQL 8.0. Connection settings are read from environment variables loaded from a local .env file; the password is not included in this report. Application queries use parameter placeholders, and request creation, assignment, status changes, and closure use transaction handling. The existing campus_maintenance database was inspected and used as the source of truth; the application does not create or replace its schema. The resulting project demonstrates how a normalized relational design can support traceable maintenance operations, consistent reporting, and a straightforward demonstration workflow for a university campus.

3. INTRODUCTION AND PROBLEM STATEMENT

3.1 Background

A campus contains buildings, rooms, equipment, and services used by students, faculty, and staff. Routine wear, electrical faults, water leaks, furniture damage, and information-technology failures require a repeatable process for reporting and resolving problems. The work is operational: a request must identify where the problem occurred, what kind of work is needed, how urgent it is, who is responsible, what work was performed, and whether the reporter considers the outcome satisfactory.

A database management system can store these related facts in a consistent structure. In this project, master data such as buildings, rooms, users, categories, priorities, technicians, and materials are stored separately from transactional records such as maintenance requests, assignments, work logs, material usage, status history, and feedback. This separation allows application pages and reports to combine the necessary information through joins without repeating every descriptive field in each request.

3.2 Existing Problem

Informal reporting can make it difficult to identify whether a fault has already been reported, which technician has accepted it, or whether it is completed or closed. A list maintained independently from work records can drift out of date. Repeating building names, room details, or technician attributes in every maintenance row also increases the risk of inconsistent spellings and update anomalies. Managers additionally need concise views of open work, technician load, response time, and material expenditure.

3.3 Problem Statement

The problem addressed is the lack of a single traceable record linking a campus maintenance issue to its reporter, location, classification, urgency, assignment, work performed, material use, status transitions, and feedback. The system must maintain relationships among these records while presenting useful operational views and rejecting invalid or duplicate data.

3.4 Proposed System

The implemented prototype is a local Flask web application connected to the existing MySQL 8.0 database named campus_maintenance. It provides a dashboard, request entry,

request search and filters, request details, technician assignment, status updates, work logs, material usage, closure, feedback, a technician listing, and reports. Request identifiers are generated by the database; a separate human-readable reference is allocated using the current year and the highest existing numeric suffix. The interface is server-rendered with Jinja2 templates and uses no front-end framework.

3.5 Need for Database Management

A relational database is appropriate because the information has explicit entities and stable relationships. Primary keys identify records independently of their display labels. Foreign keys prevent a request from referring to a nonexistent user, room, category, or priority. Unique and check constraints enforce additional rules at the data layer, even if input originates outside the web form. SQL joins and aggregate functions support building-level counts, technician workload, material cost, ageing, and feedback summaries. Transaction boundaries group related changes—for example, creating a request and its initial status-history record—so that partial writes are not committed when an operation fails.

4. OBJECTIVES AND SCOPE

4.1 Objectives

1. Centralize campus maintenance requests in the existing relational database.
2. Associate each request with a reporter, room/building, category, and priority.
3. Assign a request to a technician while respecting the one-assignment-per-request constraint.
4. Record workflow status, status history, work logs, material usage, closure details, and feedback.
5. Provide request search/filtering and a database-backed dashboard.
6. Use joins and aggregates to present useful operational reports.
7. Use environment-based database credentials and parameterized SQL.
8. Demonstrate transaction handling and server-side validation in a beginner-friendly Flask application.

4.2 Scope

The implemented scope is a local, server-rendered web prototype for recording and tracking maintenance work against the existing 13-table MySQL schema. It supports records already present in the database and form-based updates through Flask routes.

4.3 Functional Scope

Included functions are request creation and listing; text search and status/category/priority filters; joined request details; technician assignment; validated workflow status updates; work-log and material-use entry; closure with details; feedback for completed or closed requests; technician display; and SQL reports. The dashboard and reports use live query results, not hard-coded sample totals.

4.4 Limitations

The current prototype has no login screen, authentication, or role-based authorization; the application selects an existing Admin user for certain history entries. It is intended for local demonstration, not public deployment. Notifications, APIs, mobile views beyond responsive CSS, asset tracking, scheduling, and advanced analytics are not implemented. Screenshot slots in Section 11 are editable placeholders because usable screenshot image files were not available to embed.

5. SOFTWARE AND HARDWARE REQUIREMENTS

5.1 Software Requirements

Table 1: Component summary

Component	Requirement / Role
Operating system	Windows 11 development environment.
Runtime	Python 3.12.
Web framework	Flask 3.1.0.
Database server	MySQL 8.0; schema/database campus_maintenance.
Connector	mysql-connector-python 9.1.0.
Templates and UI	Jinja2, HTML5, CSS3, vanilla JavaScript.
Configuration	python-dotenv; DB_HOST, DB_USER, DB_PASSWORD, DB_NAME.
Editor and browser	Visual Studio Code and a modern web browser.

5.2 Minimum Hardware (Prototype)

Table 2: Resource summary

Resource	Reasonable minimum
Processor	Dual-core CPU capable of running Windows 11.
Memory	8 GB RAM for Windows, editor, local MySQL, and browser.
Storage	At least 2 GB free for source, dependencies, and database files.
Display	1366 × 768 or higher for comfortable form/table viewing.
Network	Localhost access; internet only for package installation/documentation.

6. ER DIAGRAM

Figure 1 is derived from SHOW CREATE TABLE and INFORMATION_SCHEMA for the live MySQL schema. It contains the actual 13 entities and shows primary keys, important attributes, foreign-key columns, and cardinalities. There is no assets table in the implementation.

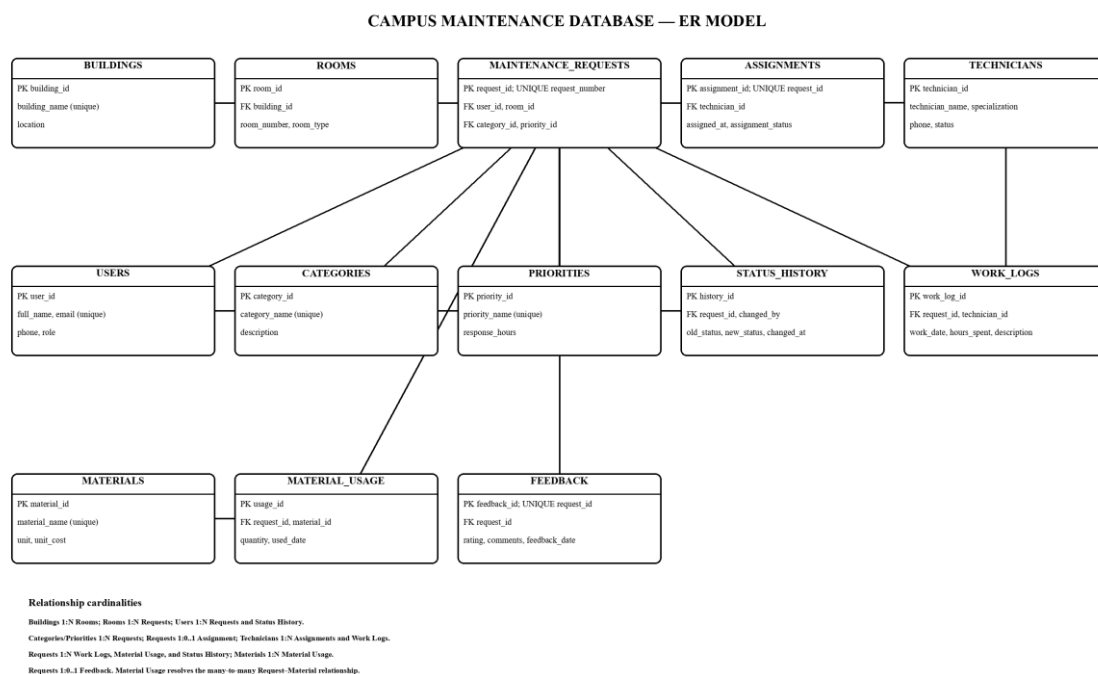


Figure 1: Entity Relationship Diagram of the Campus Maintenance Management System

6.1 Relationship Summary

Table 3: Relationship summary

Relationship	Cardinality and implementation
Building–Room	One building contains many rooms; rooms.building_id is an FK.

Relationship	Cardinality and implementation
Room–Request	One room can be referenced by many maintenance requests.
User–Request	One user can report many requests.
Category/Priority–Request	Each lookup value can classify many requests.
Request–Assignment	A request has zero or one assignment; assignments.request_id is UNIQUE.
Technician–Assignment	A technician may have multiple assignment records.
Request–Work Log	A request can have multiple technician work-log entries.
Request–Material	Many-to-many relationship resolved by material_usage.
Request–History/Feedback	History is one-to-many; feedback is zero-or-one due to UNIQUE request_id.

7. RELATIONAL SCHEMA AND NORMALIZATION

7.1 Relational Schema

Table 1: Relational Schema of the Existing Database

```

BUILDINGS(building_id PK, building_name UNIQUE, location)
ROOMS(room_id PK, building_id FK, room_number, room_type, UNIQUE(building_id, room_number))
USERS(user_id PK, full_name, email UNIQUE, phone, role CHECK)
CATEGORIES(category_id PK, category_name UNIQUE, description)
PRIORITIES(priority_id PK, priority_name UNIQUE, response_hours CHECK)
TECHNICIANS(technician_id PK, technician_name, specialization, phone UNIQUE, status CHECK)
MAINTENANCE_REQUESTS(request_id PK, request_number UNIQUE, user_id FK, room_id FK, category_id FK, priority_id FK, title, description, request_date, status, assigned_date, closed_date, closure_details)
ASSIGNMENTS(assignment_id PK, request_id FK UNIQUE, technician_id FK, assigned_at, assignment_status CHECK)
WORK_LOGS(work_log_id PK, request_id FK, technician_id FK, work_date, work_description, hours_spent CHECK)
MATERIALS(material_id PK, material_name UNIQUE, unit, unit_cost CHECK)
MATERIAL_USAGE(usage_id PK, request_id FK, material_id FK, quantity CHECK, used_date)
STATUS_HISTORY(history_id PK, request_id FK, old_status, new_status, changed_at, changed_by FK -> USERS)
FEEDBACK(feedback_id PK, request_id FK UNIQUE, rating CHECK, comments, feedback_date)

```

7.2 First Normal Form (1NF)

Each table has a primary key and stores attributes as single values for the application's data model. For example, a maintenance request stores one user_id, one room_id, one category_id, and one priority_id; multiple work entries or consumed materials are represented as separate related rows, rather than repeating groups embedded in a request record. Long descriptions and comments are text values, not lists of structured items.

7.3 Second Normal Form (2NF)

The tables use single-column surrogate primary keys, so non-key attributes depend on the whole primary key. The room uniqueness rule is a separate composite alternate key on (building_id, room_number), and its attributes describe that complete room identity.

Material quantities depend on a particular material-usage record, not on just a request or material in isolation.

7.4 Third Normal Form (3NF)

Descriptive facts about lookup entities are stored in their own relations. A room row references a building instead of repeating the building location in every request; a request references category and priority rows; a technician's name and specialization are stored with the technician rather than copied into assignments and work logs. This reduces update anomalies when a label or location changes. Separate work_logs and material_usage rows allow inserting a request without inventing work/material details and deleting a usage record without removing the material master. The schema therefore demonstrates the intended normalization benefits while retaining operational history.

8. DATA DICTIONARY

The following dictionary is generated from INFORMATION_SCHEMA for the live campus_maintenance database. Column type, nullability, defaults, auto-increment attributes, keys, and checks reflect the existing MySQL 8.0 schema. Descriptions explain the role of each field; they do not add constraints.

Table 4: Data Dictionary for the 13 Existing Tables

Column Name	Data Type	Constraint	Key	Default	Description
BUILDINGS — Campus building/location master data.					
building_id	int	NOT NULL; AUTO_INCREMENT	PK	—	Surrogate identifier for a building.
building_name	varchar(100)	UNIQUE; NOT NULL	UNIQUE	—	Unique campus building name.
location	varchar(150)	NOT NULL	—	—	Campus or physical location label.
ROOMS — Rooms belonging to a campus building.					
room_id	int	NOT NULL; AUTO_INCREMENT	PK	—	Surrogate identifier for a room.
building_id	int	REFERENCES buildings.building_id; UNIQUE (building_id, room_number); NOT NULL	FK, UK	—	Parent building foreign key.
room_number	varchar(20)	UNIQUE (building_id, room_number); NOT NULL	UK	—	Room number within its building.
room_type	varchar(50)	DEFAULT	—	Classroom	Functional room type; defaults to Classroom.
USERS — People who report requests or perform administrative actions.					

Column Name	Data Type	Constraint	Key	Default	Description
user_id	int	NOT NULL; AUTO_INCREMENT	PK	—	Surrogate identifier for a user.
full_name	varchar(100)	NOT NULL	—	—	User's display name.
email	varchar(120)	UNIQUE; NOT NULL	UNIQUE	—	Unique user email address.
phone	varchar(15)	—	—	—	Optional user phone number.
role	varchar(30)	NOT NULL; CHECK role IN (Student, Faculty, Staff, Admin); DEFAULT	—	Student	Student, Faculty, Staff, or Admin; defaults to Student.
CATEGORIES — Maintenance request classification values.					
category_id	int	NOT NULL; AUTO_INCREMENT	PK	—	Surrogate identifier for a category.
category_name	varchar(50)	UNIQUE; NOT NULL	UNIQUE	—	Unique request category label.
description	varchar(255)	—	—	—	Optional explanation of the category.
PRIORITIES — Priority levels and their response targets.					
priority_id	int	NOT NULL; AUTO_INCREMENT	PK	—	Surrogate identifier for a priority.
priority_name	varchar(30)	UNIQUE; NOT NULL	UNIQUE	—	Unique priority label.
response_hours	int	NOT NULL; CHECK response_hours > 0	—	—	Positive target response time in hours.
TECHNICIANS — Technicians and their specialization/availability.					
technician_id	int	NOT NULL; AUTO_INCREMENT	PK	—	Surrogate identifier for a technician.
technician_name	varchar(100)	NOT NULL	—	—	Technician's name.
specialization	varchar(50)	NOT NULL	—	—	Technician's maintenance specialization.
phone	varchar(15)	UNIQUE	UNIQUE	—	Optional unique technician phone number.
status	varchar(20)	CHECK status IN (Available, Busy, Inactive); DEFAULT	—	Available	Available, Busy, or Inactive; defaults to Available.
MAINTENANCE_REQUESTS — Core reported facility-maintenance incidents.					
request_id	int	NOT NULL; AUTO_INCREMENT	PK	—	Auto-increment internal primary key.
request_number	varchar(30)	UNIQUE; NOT NULL	UNIQUE	—	Unique human-readable request reference.
user_id	int	REFERENCES users.user_id; NOT NULL	FK	—	Reporting user foreign key.
room_id	int	REFERENCES rooms.room_id; NOT NULL	FK	—	Affected room foreign key.
category_id	int	REFERENCES categories.category_id; NOT NULL	FK	—	Request category foreign key.

Column Name	Data Type	Constraint	Key	Default	Description
priority_id	int	REFERENCES priorities.priority_id; NOT NULL	FK	—	Priority foreign key.
title	varchar(150)	NOT NULL	—	—	Short incident title, maximum 150 characters.
description	text	NOT NULL	—	—	Detailed report of the issue.
request_date	datetime	DEFAULT	—	CURRENT_TIMESTAMP	Report timestamp; defaults to current timestamp.
status	varchar(30)	NOT NULL; DEFAULT	—	Pending	Workflow state; defaults to Pending.
assigned_date	datetime	—	—	—	Optional time the request was assigned.
closed_date	datetime	—	—	—	Optional time the request was closed.
closure_details	varchar(500)	—	—	—	Optional closure note, maximum 500 characters.
ASSIGNMENTS — The technician assignment for a request; request_id is unique.					
assignment_id	int	NOT NULL; AUTO_INCREMENT	PK	—	Auto-increment assignment primary key.
request_id	int	REFERENCES maintenance_requests.request_id; UNIQUE; NOT NULL	FK, UNIQUE	—	Unique request foreign key, limiting each request to one assignment.
technician_id	int	REFERENCES technicians.technician_id; NOT NULL	FK	—	Assigned technician foreign key.
assigned_at	datetime	DEFAULT	—	CURRENT_TIMESTAMP	Assignment timestamp; defaults to current timestamp.
assignment_status	varchar(30)	CHECK assignment_status IN (Assigned, In Progress, Completed); DEFAULT	—	Assigned	Assignment state; defaults to Assigned.
WORK_LOGS — Technician work entries for maintenance requests.					
work_log_id	int	NOT NULL; AUTO_INCREMENT	PK	—	Auto-increment work-log primary key.
request_id	int	REFERENCES maintenance_requests.request_id; NOT NULL	FK	—	Related maintenance request foreign key.
technician_id	int	REFERENCES technicians.technician_id; NOT NULL	FK	—	Technician who recorded the work.
work_date	datetime	DEFAULT	—	CURRENT_TIMESTAMP	Work timestamp; defaults to current timestamp.
work_description	varchar(500)	NOT NULL	—	—	Work performed, maximum 500 characters.
hours_spent	decimal(5,2)	NOT NULL; CHECK hours_spent >= 0	—	—	Hours recorded to two decimal places; non-negative.
MATERIALS — Materials and their standard units and costs.					
material_id	int	NOT NULL; AUTO_INCREMENT	PK	—	Surrogate identifier for a material.

Column Name	Data Type	Constraint	Key	Default	Description
material_name	varchar(100)	UNIQUE; NOT NULL	UNIQUE	—	Unique material name.
unit	varchar(30)	NOT NULL	—	—	Unit of measurement.
unit_cost	decimal(10,2)	NOT NULL; CHECK unit_cost >= 0	—	—	Non-negative standard cost per unit.
MATERIAL_USAGE — Materials/quantities consumed by requests.					
usage_id	int	NOT NULL; AUTO_INCREMENT	PK	—	Auto-increment usage primary key.
request_id	int	REFERENCES maintenance_requests.req uest_id; NOT NULL	FK	—	Maintenance request foreign key.
material_id	int	REFERENCES materials.material_id; NOT NULL	FK	—	Material foreign key.
quantity	decimal(10,2)	NOT NULL; CHECK quantity > 0	—	—	Quantity used to two decimal places; greater than zero.
used_date	datetime	DEFAULT	—	CURRENT_ TIMESTAM P	Usage timestamp; defaults to current timestamp.
STATUS_HISTORY — Status transitions, time, and the user who changed status.					
history_id	int	NOT NULL; AUTO_INCREMENT	PK	—	Auto-increment history primary key.
request_id	int	REFERENCES maintenance_requests.req uest_id; NOT NULL	FK	—	Maintenance request foreign key.
old_status	varchar(30)	—	—	—	Prior status; NULL for an initial event.
new_status	varchar(30)	NOT NULL	—	—	Resulting status after the transition.
changed_at	datetime	DEFAULT	—	CURRENT_ TIMESTAM P	Change timestamp; defaults to current timestamp.
changed_by	int	REFERENCES users.user_id; NOT NULL	FK	—	User foreign key for the actor.
FEEDBACK — One rating/comment record per request.					
feedback_id	int	NOT NULL; AUTO_INCREMENT	PK	—	Auto-increment feedback primary key.
request_id	int	REFERENCES maintenance_requests.req uest_id; UNIQUE; NOT NULL	FK, UNIQUE	—	Unique request foreign key; one feedback row per request.
rating	int	NOT NULL; CHECK rating BETWEEN 1 AND 5	—	—	Integer rating from 1 through 5.
comments	varchar(500)	—	—	—	Optional feedback comment, maximum 500 characters.
feedback_date	datetime	DEFAULT	—	CURRENT_ TIMESTAM P	Feedback timestamp; defaults to current timestamp.

9. SQL COMMANDS USED (DDL AND DML) WITH SAMPLE OUTPUTS

9.1 Database Creation

The database named campus_maintenance was already created and populated before this Flask application was connected. The application and this report generator do not run CREATE DATABASE, DROP, TRUNCATE, ALTER, or CREATE TABLE statements. No schema changes were made for this report.

Listing 1: Read-only Database Target Verification

```
SELECT DATABASE();
-- Result observed: campus_maintenance
```

9.2 DDL Commands

There is no DDL setup script in the inspected project. The command below was used only to inspect the actual existing table definition; it is not a schema-creation instruction.

Listing 2: Existing DDL Inspection Command (Read Only)

```
SHOW CREATE TABLE maintenance_requests;
```

The inspected definition has request_id as AUTO_INCREMENT PRIMARY KEY; UNIQUE request_number; foreign keys to users, rooms, categories, and priorities; DEFAULT CURRENT_TIMESTAMP for request_date; and DEFAULT 'Pending' for status. The complete per-column schema, including the other 12 tables, appears in Section 8.

9.3 DML Commands

Listing 3: Parameterized Read and Insert Patterns Used by Flask

```
cursor.execute(
    "SELECT user_id, full_name AS name FROM users ORDER BY full_name"
)

cursor.execute(
    """
    INSERT INTO maintenance_requests
    (request_number, title, description, user_id, room_id, category_id, priority_id)
    VALUES (%s, %s, %s, %s, %s, %s, %s)
    """,
    (request_number, title, description, user_id, room_id, category_id, priority_id),
)
```

9.4 Constraints

The existing schema enforces primary keys, foreign keys, UNIQUE constraints, NOT NULL requirements, defaults, and CHECK constraints. Notable examples are the unique request_number; unique assignments.request_id; unique feedback.request_id; (building_id, room_number) uniqueness for rooms; role/status domain checks; positive response_hours and material quantity; non-negative hours and material cost; and a feedback rating from 1 to 5.

9.5 Sample Data and Observed Output

At report preparation, the live database contained 7 maintenance requests, 5 technicians, and 2 feedback entries. The following five seeded sample records are shown with their actual internal IDs. They are not IDs 1–5.

Table 5: request_id summary

request_id	Request Number	Title	Status
11	REQ-2026-001	Broken classroom light	Pending
12	REQ-2026-002	Damaged classroom chair	Assigned
13	REQ-2026-003	Computer not working	In Progress
14	REQ-2026-004	Water leakage	Completed
15	REQ-2026-005	Faulty switch	Closed

10. QUERIES WITH OUTPUTS

These examples are the same query patterns implemented by the application's request listing and Reports page. Output samples below are read from the live database on the report-generation date; changing data later will change the output.

10.1 Main Request Report (JOIN)

Listing 4: Request List JOIN

```
SELECT mr.request_id, mr.request_number, mr.title,
       u.full_name AS reported_by, c.category_name, p.priority_name,
       mr.status, b.building_name, r.room_number
FROM maintenance_requests mr
LEFT JOIN users u ON u.user_id = mr.user_id
LEFT JOIN rooms r ON r.room_id = mr.room_id
LEFT JOIN buildings b ON b.building_id = mr.building_id
LEFT JOIN categories c ON c.category_id = mr.category_id
LEFT JOIN priorities p ON p.priority_id = mr.priority_id
ORDER BY mr.request_date DESC;
```

Table 6: ID summary

ID	Request	Title	Status	Building / Room
11	REQ-2026-001	Broken classroom light	Pending	Main Academic Block / 101
12	REQ-2026-002	Damaged classroom chair	Assigned	Main Academic Block / 102
13	REQ-2026-003	Computer not working	In Progress	Engineering Block / 301

The join obtains reporter, location, category, and priority labels without duplicating those descriptions in maintenance_requests. IDs 11–15 map directly to the five original request numbers.

10.2 Pending Requests

Listing 5: Pending Request Report

```
SELECT mr.request_number, mr.title, c.category_name, p.priority_name,
       b.building_name, r.room_number, mr.status
FROM maintenance_requests mr
LEFT JOIN rooms r ON r.room_id = mr.room_id
LEFT JOIN buildings b ON b.building_id = r.building_id
LEFT JOIN categories c ON c.category_id = mr.category_id
LEFT JOIN priorities p ON p.priority_id = mr.priority_id
WHERE LOWER(COALESCE(mr.status, '')) = 'pending'
ORDER BY mr.request_date DESC;
```

Table 7: Request Number summary

Request Number	Title	Category	Priority	Building / Room
REQ-2026-007	AC is not working	Electrical	Critical	Administrative Block / A101
REQ-2026-006	Automated app verification request	Electrical	High	Main Academic Block / 101
REQ-2026-001	Broken classroom light	Electrical	High	Main Academic Block / 101

10.3 Technician Workload

Listing 6: Technician Load Aggregate

```
SELECT tech.technician_name, tech.specialization,
       COUNT(a.assignment_id) AS assigned_requests
FROM technicians tech
LEFT JOIN assignments a ON a.technician_id = tech.technician_id
GROUP BY tech.technician_id, tech.technician_name, tech.specialization
ORDER BY assigned_requests DESC;
```

Table 8: Technician summary

Technician	Specialization	Requests
Amit Singh	Furniture	1
Ramesh Kumar	Electrical	1
Suresh Patel	Plumbing	1
Vijay Rao	IT Equipment	1

Technician	Specialization	Requests
Mahesh Yadav	Civil	0

10.4 Material Cost and Building Counts

Listing 7: Material Quantity and Cost Aggregate

```
SELECT m.material_name, SUM(mu.quantity) AS quantity_used,
       m.unit_cost, SUM(mu.quantity * m.unit_cost) AS total_cost
FROM material_usage mu
LEFT JOIN materials m ON m.material_id = mu.material_id
GROUP BY m.material_id, m.material_name, m.unit_cost
ORDER BY total_cost DESC;
```

Table 9: Material summary

Material	Quantity	Unit Cost	Total Cost
Computer Mouse	1.00	500.00	500.00
Water Tap	1.00	350.00	350.00
PVC Pipe	2.00	120.00	240.00
Switch	1.00	80.00	80.00

Table 10: Building summary

Building	Request Count
Main Academic Block	3
Engineering Block	2
Administrative Block	1
Library Block	1

10.5 Feedback, Ageing, and Response Time

Feedback is joined to its request number and ordered by feedback_date. Request ageing uses DATEDIFF(NOW(), request_date) for requests not in Completed or Closed status. Response time uses DATEDIFF(assigned_at, request_date) where an assignment exists. In the sampled rows, ages and response intervals were zero whole days because the records were created/assigned on the same date; DATEDIFF intentionally reports calendar-day differences, not hours.

Table 11: Request summary

Request	Status	Age (days)
REQ-2026-001	Pending	0
REQ-2026-002	Assigned	0
REQ-2026-003	In Progress	0
REQ-2026-006	Pending	0
REQ-2026-007	Pending	0

Table 12: Feedback request summary

Feedback request	Rating	Comments
REQ-2026-004	5	Issue was resolved quickly and professionally.
REQ-2026-005	4	Good service and quick response.

Table 13: Assigned request summary

Assigned request	Response (days)
REQ-2026-002	0
REQ-2026-003	0
REQ-2026-004	0
REQ-2026-005	0

11. UI DESIGN AND SCREENSHOTS

The screenshots shown in this section are intentionally editable placeholders, not fabricated captures. The local application provides the pages represented below; replace each box with a screenshot captured from the running browser before final submission.

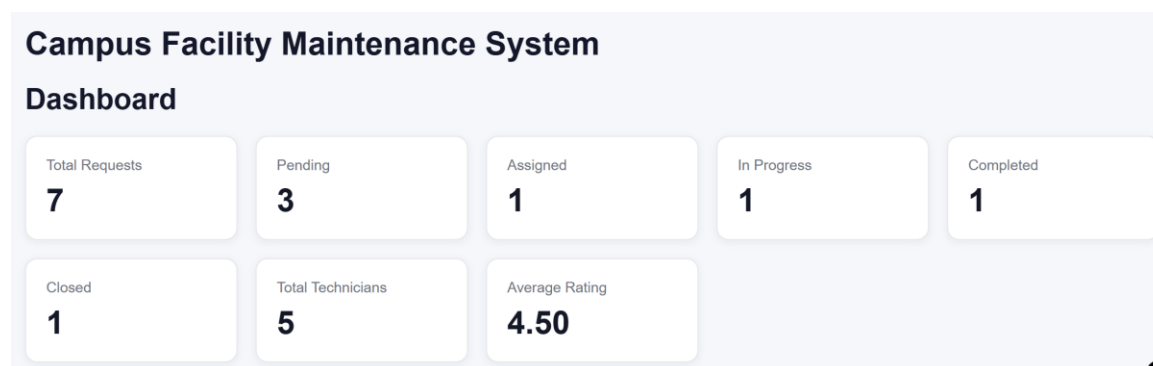


Figure 2: Dashboard

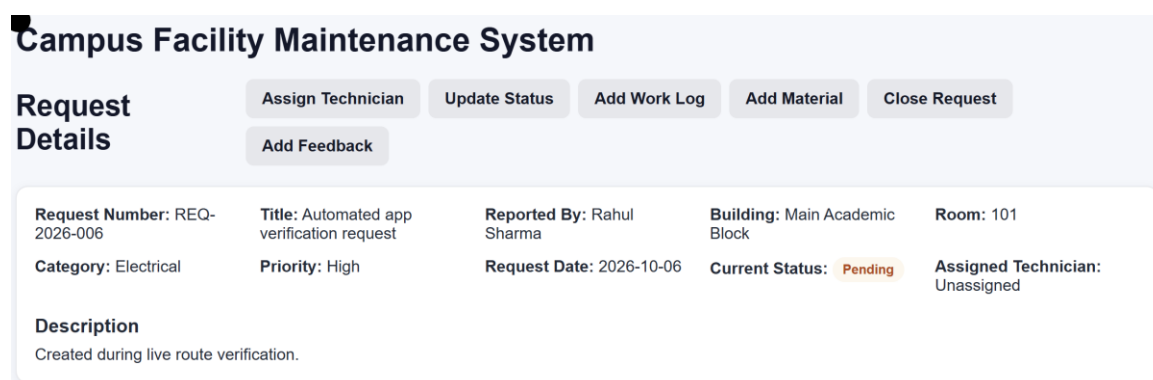


Figure 3: Request Details

Campus Facility Maintenance System

Create Maintenance Request

User

Room

Category

Select user

Select room

Select category

Priority

Select priority

Title

Description

Submit Request

Cancel

Figure 4: Create Maintenance Request

Request List									
Request Number	Title	User	Building	Room	Category	Priority	Status	Request Date	Actions
REQ-2026-007	AC is not working	Arjun Kumar	Administrative Block	A101	Electrical	Critical	Pending	2026-10-06	View
REQ-2026-006	Automated app verification request	Rahul Sharma	Main Academic Block	101	Electrical	High	Pending	2026-10-06	View
REQ-2026-001	Broken classroom light	Rahul Sharma	Main Academic Block	101	Electrical	High	Pending	2026-10-06	View
REQ-2026-002	Damaged classroom chair	Priya Reddy	Main Academic Block	102	Furniture	Medium	Assigned	2026-10-06	View
REQ-2026-003	Computer not working	Arjun Kumar	Engineering Block	301	IT Equipment	Critical	In Progress	2026-10-06	View
REQ-2026-004	Water leakage	Sneha Rao	Library Block	L101	Plumbing	High	Completed	2026-10-06	View
REQ-2026-005	Faulty switch	Rahul Sharma	Engineering Block	302	Electrical	Low	Closed	2026-10-06	View

Figure 5: Maintenance Request List

Technicians		
Name	Specialization	Status
Amit Singh	Furniture	Available
Mahesh Yadav	Civil	Busy
Ramesh Kumar	Electrical	Available
Suresh Patel	Plumbing	Busy
Vijay Rao	IT Equipment	Available

Figure 6: Technician Assignment

12. IMPLEMENTATION DETAILS

12.1 Technology Stack

Python 3.12 and Flask implement server-side routes. mysql-connector-python supplies MySQL access. Jinja2 renders the HTML templates; HTML5 and CSS3 provide forms, tables, responsive layout, and status badges. A small vanilla JavaScript file defaults blank date inputs to the current date. Dependencies are listed in requirements.txt. No APIs, client-side framework, or authentication subsystem are present.

12.2 System Architecture

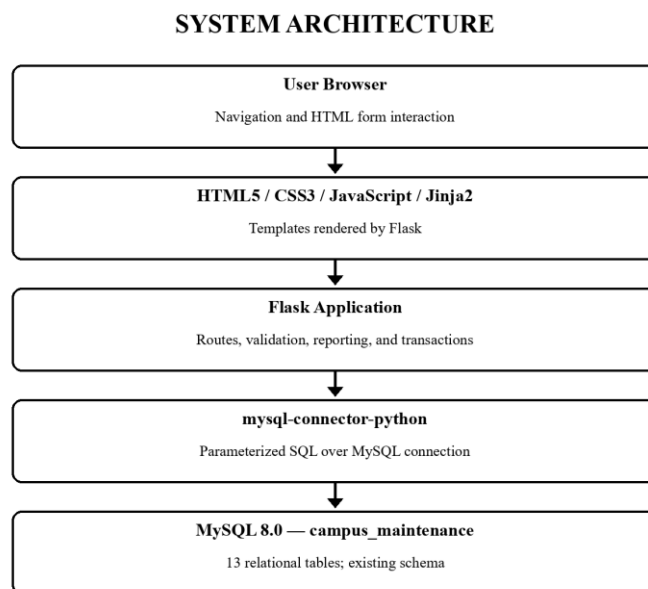


Figure 12: System Architecture

12.3 Frontend Implementation

The shared base template defines navigation links to Dashboard, Requests, Create Request, Technicians, and Reports. Child templates provide request forms, joined detail views, tables, and report sections. The stylesheet defines the sidebar, cards, panels, badges, table overflow handling, and a narrow-screen layout. Flask's Jinja environment autoescapes template values.

12.4 Flask Backend and Routes

The route functions retrieve request records and lookups, validate submitted values, invoke parameterized SQL, and render templates or redirect after successful writes. The request list supports search by request number/title and filters by status, category, and

priority. The detail page queries assignments, status history, work logs, material usage, and feedback. Database errors are logged to the Flask terminal and returned as friendly page messages.

12.5 MySQL Connectivity and Configuration

config.py loads a local .env through python-dotenv and maps DB_HOST, DB_USER, DB_PASSWORD, and DB_NAME to mysql.connector.connect. The project .gitignore excludes .env. The example file contains placeholders only. Reusable fetch_one/fetch_all helpers open a dictionary cursor, execute SQL, close the cursor and connection in finally blocks, and allow database exceptions to reach application error handling.

Listing 8: Database Connection and Environment-Based Settings

```
def get_db_connection():
    try:
        return mysql.connector.connect(**get_db_config())
    except Error:
        app.logger.exception("Could not connect to the configured MySQL database")
        raise
```

12.6 CRUD, Validation, and Transactions

Create request is the principal create operation and uses request/user/room/category/priority data. Read operations power the dashboard, request list, detail view, technician page, and reports. Updates change status, assigned dates, closure fields, and technician state; inserts record assignments, work logs, material use, history, and feedback. Form validation checks required fields, numeric hours, positive material quantity, rating range, and allowed status transitions. Duplicate assignment and feedback prevention rely on the schema's UNIQUE constraints.

Listing 9: Request Creation Transaction and Initial History

```
conn.start_transaction()
request_number = next_request_number(cursor)
cursor.execute(
    """
    INSERT INTO maintenance_requests
    (request_number, title, description, user_id, room_id, category_id, priority_id)
    VALUES (%s, %s, %s, %s, %s, %s, %s)
    """
    ,
    (request_number, title, description, user_id, room_id, category_id, priority_id),
)
request_id = cursor.lastrowid
cursor.execute(
    "INSERT INTO status_history (request_id, old_status, new_status, changed_by) "
    "VALUES (%s, NULL, 'Pending', %s)",
    (request_id, user_id),
)
```

```
)  
conn.commit()
```

Listing 10: Parameterized Search and Status Update

```
if search:  
    query += " AND (mr.request_number LIKE %s OR mr.title LIKE %s)"  
    like = f"%{search}%"  
    params.extend([like, like])  
    cursor.execute(query, tuple(params))  
  
cursor.execute(  
    "UPDATE maintenance_requests SET status = %s, closed_date = %s, "  
    "closure_details = %s WHERE request_id = %s",  
    (new_status, closed_date, closure_details, request_id),  
)
```

12.7 Reporting and Workflow

Reports use COUNT, SUM, DATEDIFF, joins, and GROUP BY for pending and ageing requests, technician workload, building request count, material usage and cost, building material cost, feedback, and response time. The following diagram summarizes the implemented user workflow; it is a process view, not an additional database entity model.

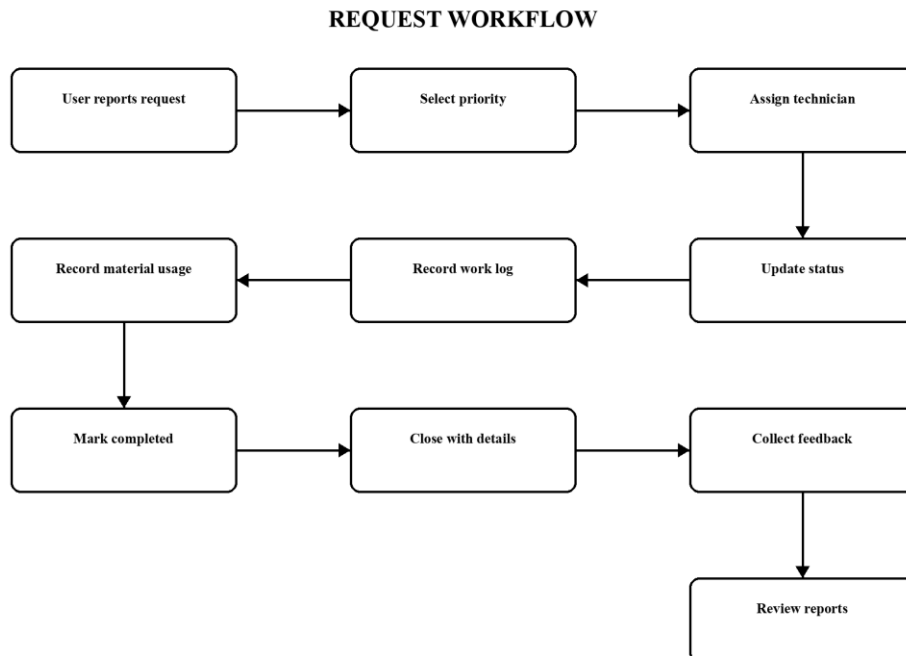


Figure 13: Application Workflow Diagram

13. TESTING

The table distinguishes checks exercised during report preparation from write paths not exercised in order to avoid adding or altering project records while preparing a document. PASS is used only for checks with observed results.

Table 14: Test ID summary

Test ID	Test Case	Input / Action	Expected Result	Observed Result	Status
TC01	Database connection	Connect with .env config.	Connect to campus_maintenance.	Connection and SELECT succeeded.	PASS
TC02	Dashboard	GET /.	Show live counts and recent rows.	HTTP 200; live values shown.	PASS
TC03	Existing requests	GET /requests.	Show original five requests.	IDs 11–15; REQ-2026-001...005 shown.	PASS
TC04	Create request	POST valid fields.	Insert request and initial history.	Form insert verified in MySQL.	PASS
TC05	Request number	Read current max suffix.	Generate next year-prefixed reference.	006 inserted; next number was 007.	PASS
TC06	Search	Search REQ-2026-001.	Show matching request only.	Matching result verified.	PASS
TC07	Filter	Filter status=Closed.	Show closed, exclude pending.	REQ-2026-005 matched.	PASS
TC08	Assignment	POST technician choice.	Atomically save assignment and history.	GET verified; POST not exercised.	NOT RUN
TC09	Status update	POST allowed next state.	Save status and history.	Valid POST not exercised.	NOT RUN
TC10	Work log	POST description and hours.	Insert work entry.	GET verified; POST not exercised.	NOT RUN
TC11	Material usage	POST material and quantity.	Insert usage for report totals.	GET verified; POST not exercised.	NOT RUN
TC12	Completion	In Progress → Completed.	Update request and assignment.	Not exercised; live data preserved.	NOT RUN
TC13	Closure	Close with details.	Save status, date, details, history.	Not exercised; live data preserved.	NOT RUN
TC14	Feedback	Submit rating 1–5.	Accept once for eligible request.	POST not exercised.	NOT RUN
TC15	Reports	GET /reports.	Render SQL report sections.	HTTP 200; outputs verified.	PASS
TC16	Invalid input	POST empty create form.	Reject without insert.	Validation returned; count unchanged.	PASS
TC17	Duplicate feedback	POST feedback for rated request.	UNIQUE key rejects duplicate.	Not submitted during report prep.	NOT RUN
TC18	Invalid transition	Pending → Closed.	Reject before database update.	Transition rejected.	PASS

14. CONCLUSION AND FUTURE ENHANCEMENTS

14.1 Conclusion

The project demonstrates a working database-backed workflow for campus facility maintenance. Its MySQL design separates the principal entities, uses keys and domain constraints to protect consistency, and records assignment, work, material, status-history, and feedback details. Flask routes provide a straightforward interface to create, search, inspect, and report on requests. Live validation confirmed the database connection, the request joins, seeded sample IDs, dashboard, dropdowns, reports, search/filter behavior,

and creation of a subsequent request number. The report describes the existing database rather than recreating it.

14.2 Future Enhancements (Not Implemented)

The following items are proposals only and are not part of the current prototype: user authentication; role-based access control; email/SMS notifications; a dedicated mobile application; advanced analytics; automated priority prediction; technician location tracking; asset management; maintenance scheduling; and cloud deployment. These additions would require separate requirements, threat analysis, data design, and testing.

15. REFERENCES

- [1] Python Software Foundation. (n.d.). Python 3.12 documentation. <https://docs.python.org/3.12/>
- [2] Pallets Projects. (n.d.). Flask documentation. <https://flask.palletsprojects.com/>
- [3] Oracle. (n.d.). MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/>
- [4] Oracle. (n.d.). MySQL Connector/Python Developer Guide. <https://dev.mysql.com/doc/connector-python/en/>
- [5] Pallets Projects. (n.d.). Jinja documentation. <https://jinja.palletsprojects.com/>
- [6] WHATWG. (Living Standard). HTML: The Living Standard. <https://html.spec.whatwg.org/>
- [7] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill.

16. CONTRIBUTION OF EACH MEMBER

The repository does not identify authorship by individual. The allocation below is a balanced editable draft for the team to review and correct before submission; it is not independently verifiable from source files.

Table 15: Work Area summary

S. No.	Team Member	Contribution
1	Pranav Arjun Nikam	Database design and implementation, SQL query development, Flask–MySQL connectivity, backend integration, system testing, and final project integration.
2	Purva Derkar	Requirement analysis, database documentation, UI design support, report preparation, testing, and validation of system functionality.
3	Sharanya Mishra	Frontend development support, user interface testing, SQL output verification, documentation, and preparation of project presentation materials.
4	Saathvik Mahalsa	Database schema support, sample data preparation, functionality testing, report documentation, and overall project integration support.

17. APPENDIX: GITHUB REPOSITORY LINK

17.1 GitHub Repository

<https://github.com/pranavv-tech/DBMS-Course-Project.git>

17.2 Project Folder Structure

Listing 11: Inspected Project Structure

```

campus_maintenance/
├── app.py; config.py; requirements.txt; .env.example; .gitignore
├── templates/ (base, dashboard, requests, details, forms, reports, errors)
├── static/ (css/style.css; js/script.js)
└── report_assets/ (ER, architecture, workflow diagrams; report generator)

```

17.3 Report Preparation Notes

Prepared on 2026-10-06 from the current project source and a read-only inspection of MySQL. The live database reported 7 maintenance rows at generation time; the five original sample requests retain internal request IDs 11, 12, 13, 14, and 15. The local .env

password is not reproduced here. No database schema or records were changed to generate this report.